

1.1 Lesson: Deep Learning, Neural Networks, and Artificial Neurons



Deep Learning, Neural Networks, and Artificial Neurons

What is Deep Learning

Deep learning is a subset of supervised learning that utilizes artificial neural networks (ANNs) to model complex relationships in data. These networks are composed of layers of computational units called **artificial neurons**, which are loosely inspired by biological neurons in the brain. Each neuron processes numerical inputs and transmits a signal to other neurons in the network.

Structure of a Neural Network

A typical neural network is structured in layers:

- **Input Layer:** While often referred to as a “layer,” the input layer isn't made up of neurons; it is simply the raw inputs to the network (a vector of floating-point numbers).
- **Hidden Layers:** Intermediate layers of neurons that process the inputs using weighted connections and activation functions.
- **Output Layer:** The final layer of neurons that produces the network's output, also in the form of a vector.

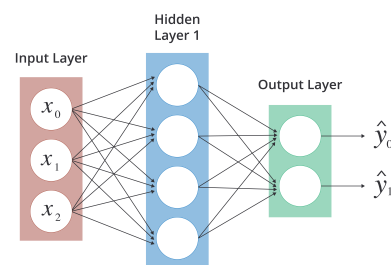


Figure 1: Artificial Neural Network with Three Layers

Each layer in the network transforms its input vector into a new vector, ultimately mapping the input data to an output. As such, neural networks are *universal vector function approximators*, which in principle can implement any function from input vectors to output vectors.

How Neurons Work

Each neuron performs **logistic regression** by calculating the **dot product** of inputs $\mathbf{x}$ and weights $\mathbf{w}$:

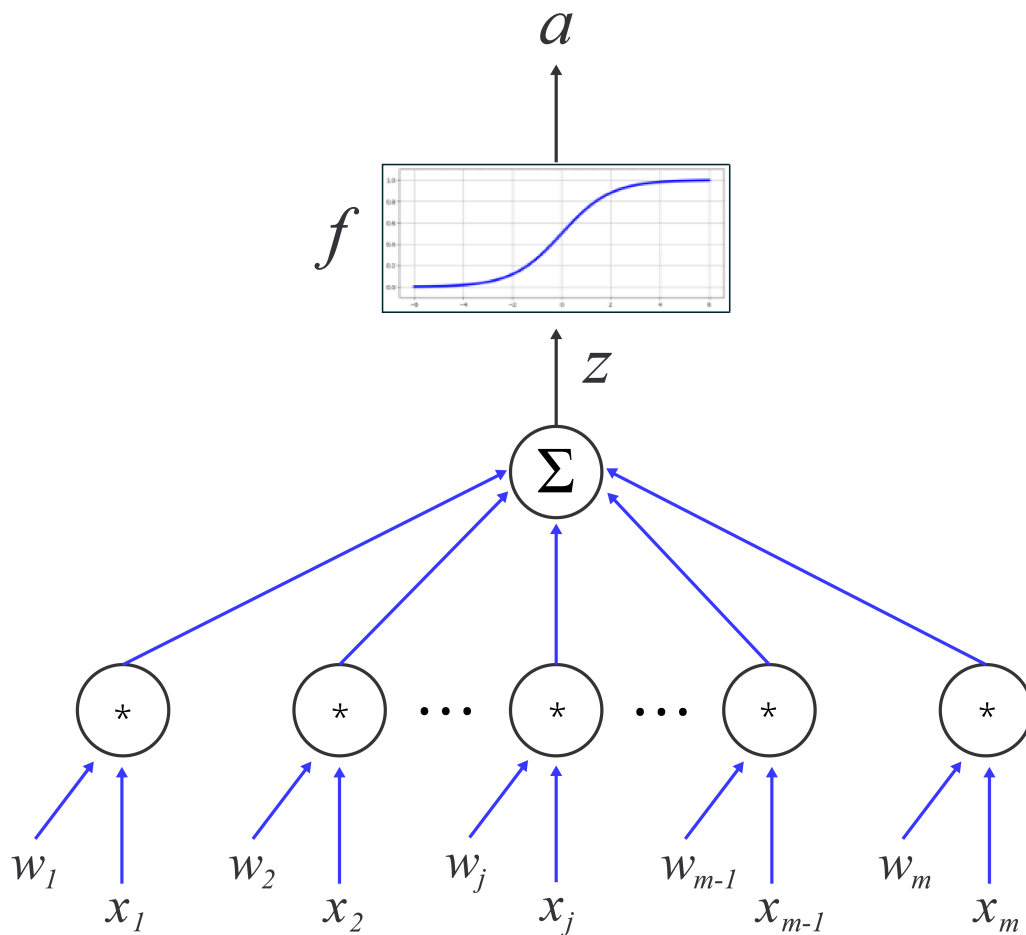


Figure 2: Logistic Regression Neuron

Using linear algebra notation, we can think of this as taking the **dot product** of inputs $\mathbf{x}$ and weights $\mathbf{w}$:

$$\mathbf{w} \cdot \mathbf{x} = (w_1 \cdot x_1) + \dots + (w_k \cdot x_k) = \mathbf{w}^T \mathbf{x}.$$

and then applying the activation function f :

$$a = f(\mathbf{w}^T \mathbf{x})$$

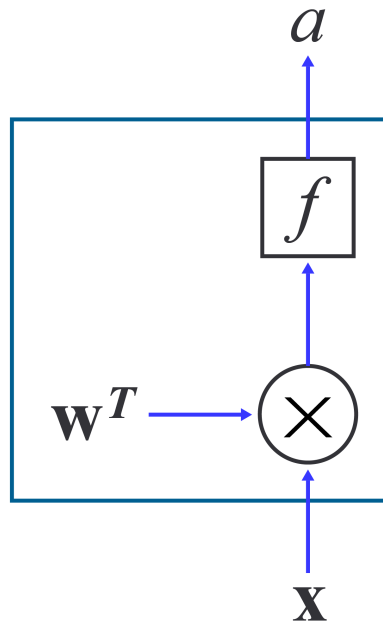


Figure 3: *Neuron in Linear Algebra Form*

A small but important detail: We have so far obscured the fact that one of the inputs to the neuron is a bias term b (as in the equation for a line: $y = mx + b$). A convenient way to encode this is to assume that the input vector to all neurons contains a constant 1.0, and so $w_m = b$.

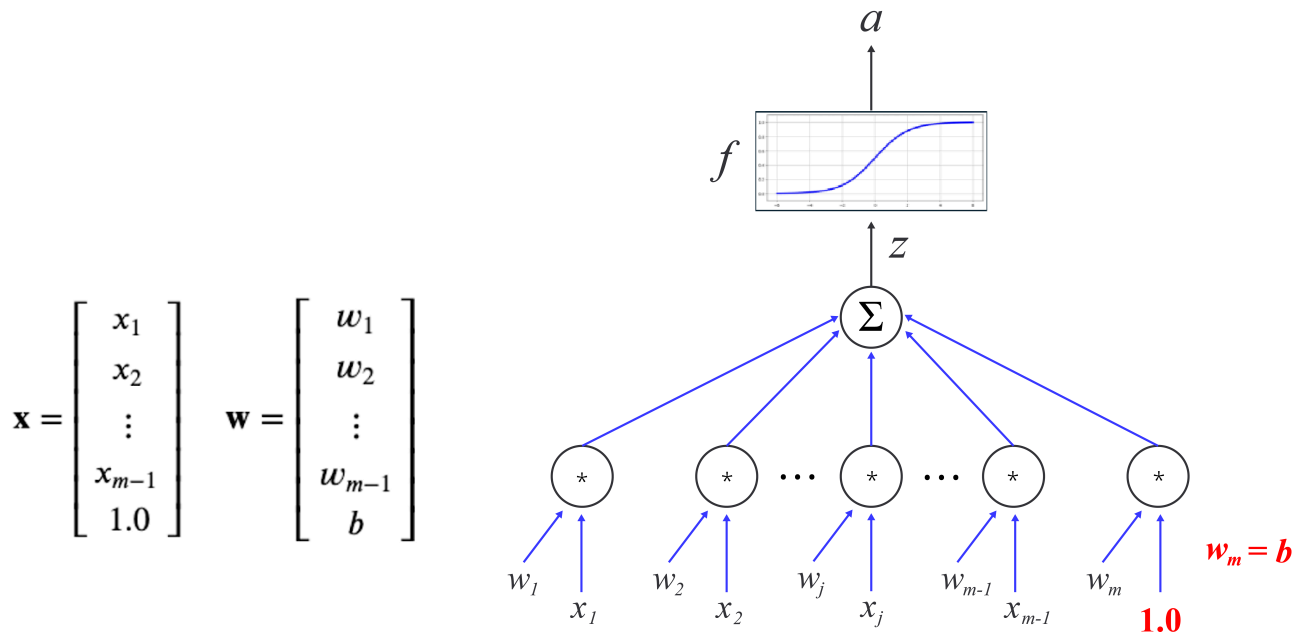


Figure 4: Encoding the Bias Term as a Weight

Activation Functions

The activation function f introduces non-linearity into the network, allowing it to learn complex patterns. Common activation functions include:

1) Sigmoid Function: Maps any input to a range between 0 and 1, useful for probability-based outputs.

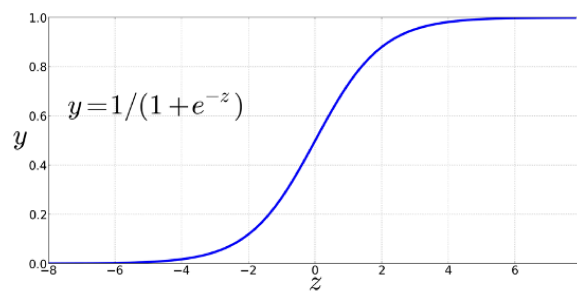


Figure 5: Sigmoid Function

2) Tanh Function: Similar to sigmoid but maps values to the range -1 to 1, often leading to better performance in large networks.

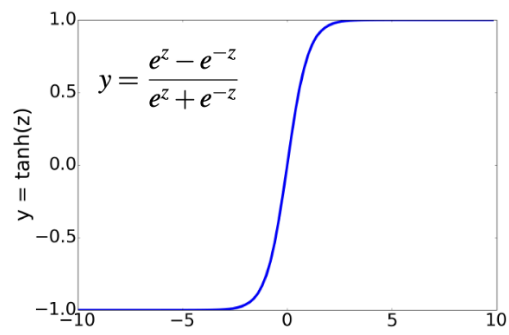


Figure 6: *Tanh Function*

3) ReLU (Rectified Linear Unit): Allows only positive values, making it computationally efficient and effective for deep networks.

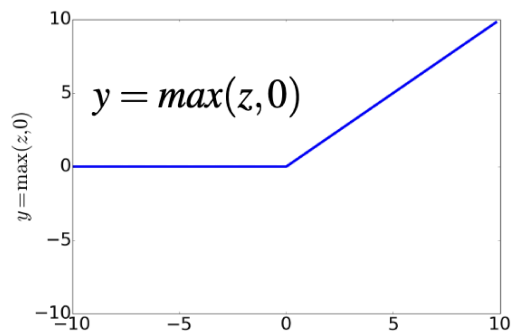


Figure 7: *ReLU Function*